



RS6x 7x 毫米波雷达 Sample 参考手册

文档页数：39

文档版本号：1.1

发布时间：2026-8-13

版本历史

版本	版本描述	日期
1.1	表 3-3 中添加 6241 默认配置说明	2026-08-13
1.0	初版建立	2026-06-12

目录

版本历史.....	ii
目录.....	iii
图片目录.....	vi
表格目录.....	vii
1 目的和应用范围.....	1
2 概述.....	2
3 体验基于 radar framework 的 Sample.....	3
3.1 选择 Sample 进行体验.....	3
3.2 设置硬件.....	4
3.3 查看 Sample 输出.....	4
4 使用 radar framework 进行二次开发.....	7
4.1 代码结构.....	7
4.2 配置 radar framework.....	8
4.2.1 功能配置指南.....	8
4.2.2 radar framework 框架全局配置.....	9
4.2.2.1 全局使能 1D 帧结构.....	9
4.2.2.2 全局使能 2D 帧结构.....	9
4.2.2.3 配置最大支持的距离 FFT 点数.....	9
4.2.2.4 配置 DataCube 缓冲区.....	9
4.2.2.5 配置自适应增益.....	9
4.2.2.6 上报帧信息.....	10
4.2.2.7 使能心跳包上报.....	10
4.2.2.8 使能看门狗.....	10
4.2.2.9 使能 Task Monitor.....	10
4.2.3 1D 帧结构子功能配置.....	10
4.2.3.1 上报 1D-DataCube.....	11
4.2.3.2 占用堆配置.....	11
4.2.4 2D 帧结构功能配置.....	11
4.2.4.1 2D 帧结构全局使能.....	11
4.2.4.2 上报 2D-DataCube.....	12
4.2.5 获取 Interval0 的距离 FFT.....	12
4.2.6 移动点云.....	13
4.2.6.1 配置软件 CFAR 功能.....	13
4.2.6.2 配置上报移动点云.....	13

4.2.6.3 配置移动点云最大 CFAR 点数.....	13
4.2.6.4 移动点云堆配置.....	13
4.2.7 存在点云.....	14
4.2.7.1 配置存在点云最大点数.....	14
4.2.7.2 配置存在点云 CFAR 模式.....	14
4.2.7.3 配置存在点云的距离 Bin 个数.....	14
4.2.7.4 配置存在点云距离抽取系数.....	14
4.2.7.5 配置存在点云缓冲 Interval 数.....	15
4.2.7.6 配置存在点云多普勒 FFT 点数.....	15
4.2.7.7 存在点云堆配置.....	15
4.3 基于 radar framework 的应用配置.....	15
4.3.1 波形配置.....	16
4.3.1.1 MIMO 模式.....	16
4.3.1.2 起始频率.....	16
4.3.1.3 帧类型.....	16
4.3.1.4 距离 FFT 点数(log2).....	17
4.3.1.5 距离分辨率.....	17
4.3.1.6 多普勒 FFT 点数(log2).....	17
4.3.1.7 速度分辨率.....	17
4.3.1.8 Chirp 累积数(log2).....	18
4.3.1.9 FFT 模式.....	18
4.3.1.10 静态杂波收敛模式.....	18
4.3.2 点云性能配置指南.....	19
4.3.3 功能裁剪指南.....	19
4.3.4 典型波形配置.....	19
4.3.5 典型性能指标.....	20
4.4 数字信号处理框架介绍.....	21
4.5 HIF 配置框架介绍.....	22
5 FAQ.....	23
5.1 如何提升点云密度?	23
5.2 如何提升可用的 SRAM?	23
5.3 为何点云输出会卡顿, 如何减少计算时间?	23
5.4 波形配置有没有最佳实践?	24
5.5 计算时间有什么约束?	24
5.6 如何提升雷达视野内对微弱反射目标的探测能力?	25
5.7 为什么容易在近距离处存在点云有虚检?	25
5.8 为什么移动点云在室内有很多假目标?	25

5.9 为什么使用 RadarDebugTool 配置可能失败?26

5.10 为什么有的 Chirp 累积数下频谱和点云会异常? 26

5.11 调试串口打印 malloc fail 怎么办? 26

5.12 调试串口打印的错误信息有什么含义?26

6 附录.....28

6.1 使用 ADC 和 1D-DataCube 数据进行数字信号处理..... 28

6.1.1 速度补偿系数..... 28

6.1.2 角度校准值..... 28

6.1.3 ADC 数据点云计算流程..... 28

图片目录

图 3-1	上位机显示点云.....	5
图 3-2	上位机展示 DataCube.....	6
图 4-1	配置参数覆盖逻辑.....	8
图 4-2	1D 帧结构下的数据通路.....	11
图 4-3	2D 帧结构下的数据通路.....	12
图 4-4	波形结构示意图.....	16
图 4-5	2D 数字信号处理框架.....	22
图 4-6	HIF 配置框架调用流程.....	22
图 5-1	多径干扰.....	25
图 5-2	“影子”假目标.....	26
图 6-1	ADC 数据点云计算流程图.....	31

表格目录

表 3-1 示例工程基本功能.....	3
表 3-2 示例工程应用场景.....	3
表 3-3 默认硬件设置.....	4
表 4-1 典型计算时间.....	20
表 4-2 可用 SRAM 大小.....	20

1 目的和应用范围

本文档旨在详细描述如何使用、配置正和微芯 RS6x_7x 系列毫米波雷达 SDK 提供的雷达相关的 Sample。开发者可在充分了解这些 Sample 的基础上基于此 Sample 框架定制或优化算法，新增后处理逻辑，从而快速进行产品开发。

本文档支持的芯片包括：RS6x 和 RS7x 系列所有芯片。

2 概述

毫米波雷达 SDK 中提供的 Sample 基于 radar framework 框架，它是一套轻量级、可配置的雷达框架，可实现下列雷达应用相关功能：

- 支持以 1D 或 2D 帧结构传输 DataCube 数据，并在 RadarDebugTool 中进行可视化；
- 支持以 1D 或 2D 帧结构传输 ADC 数据，并在 RadarDebugTool 中进行可视化；
- 支持在 2D 帧结构下上报第一个 Interval 的 RangeFFT 数据，并在 RadarDebugTool 中进行可视化；
- 支持移动点云计算和上报，并在 RadarDebugTool 中进行可视化；
- 支持存在点云的计算和上报，并在 RadarDebugTool 中进行可视化；
- 支持通过 RadarDebugTool 配置基础的波形参数；
- 支持产测功能，并可通过 ProductTestingAnalysisTool 进行测试；
- 支持休眠唤醒功能，在 CPU 完成数字信号处理后进入休眠状态，功耗<10uA；
- 支持在雷达数字信号处理开始和结束时调用钩子函数；
- 支持自定义 HIF 应用层传输和配置协议。

radar framework 框架经过了稳定性和压力测试，并具备保活机制，可作为量产的基础版本进行二次开发。

3 体验基于 radar framework 的 Sample

SDK 提供了预先编译好的固件文件，开发者可直接根据需求选择固件文件下载至开发板中，使用 RadarDebugTool 工具快速评估基于 radar framework 的 DataCube、移动点云和存在点云性能，使用 ProductTestingAnalysisTool 验证产测功能。

固件目录位于：

[/Software Kit/04 Image/Tracking \[board name\].img。](#)

[/Software Kit/04 Image/Posture \[board name\].img。](#)

[/Software Kit/04 Image/Sleep \[board name\].img。](#)

[/Software Kit/04 Image/ReportDataCube1D \[board name\].img。](#)

[/Software Kit/04 Image/ReportDataCube2D \[board name\].img。](#)

关于 RadarDebugTool 的使用和固件下载的方法，请参考《RS6x_7x_毫米波雷达开发_快速入门》。

3.1 选择 Sample 进行体验

各种示例工程支持的 radar framework 的功能如下，固件以[功能]_[开发板型号].img 命名，下载固件前请务必确定固件对应的开发板与使用的开发板型号对应。若不清楚开发板型号，请参考《RS6x_7x 毫米波雷达开发_快速入门》。Sample 的具体功能将在 4.2 中描述。

表 3-1 示例工程基本功能

工程名	1D-Dat aCube	2D-DataC ube	移动 点云	存在 点云	Interval0 RFFT	帧周期	产测	低功耗
ReportDataCu be1D	√	×	×	×	×	10ms	×	×
ReportDataCu be2D	×	√	×	×	×	100ms	×	×
Posture	×	×	√	√	×	50ms	√	×
Sleep	×	×	√	√	√	50ms	√	×
Tracking	×	×	√	×	×	50ms	√	√

各种 Sample 对应的典型应用场景如下表所示：

表 3-2 示例工程应用场景

	应用场景
--	------

	应用场景
ReportDataCube1D	雷达作为 1D 帧结构数据发生器，最高支持 200Hz 的 Interval 周期，通过 SPI 将数据发送给上位机，在上位机上完成后续数字信号处理。
ReportDataCube2D	雷达作为 2D 帧结构数据发生器，通过 SPI 将数据发送给上位机，在上位机上完成后续数字信号处理。
Posture	用于 5m 以内近距离姿态识别应用和机器人平台，产生近距离、高密度的移动和存在点云，不进行静止目标的探测。
Sleep	用于 10 米以内中距离、中等密度移动和存在点云。支持上报 Interval0 的距离 FFT 数据，用于呼吸心跳检测应用进行后级处理，不进行静止目标的探测。
Tracking	用于 20 米以内，户外场景下，支持低功耗特性的移动点云，不进行静止目标的探测。

3.2 设置硬件

针对不同开发板编译的 Sample 固件具有不同的默认通信接口。请首先参考《RS6x_7x_毫米波雷达评估_快速入门》中“硬件环境搭建”章节了解不同通信模式的定义，并完成硬件设定。

表 3-3 默认硬件设置

	MRS6130_P1806	MRS6130_P1812	MRS6240_P2512/ MRS6241_P2828/ MRS6241_P2840
通信模式	DUAL-UART	UART-SPI	UART-SPI
调试信息打印	UART1, 115200bps	UART, 115200bps	UART, 115200bps
HIF（数据接口）	UART0, 1Mbps	SPI, 60Mbps	SPI, 60Mbps

注意，体验 Sample 功能时请务必将雷达放置于稳定平台，移动雷达过程中可能导致点云密度过大计算超时，影响评估。平台产生的非预期的震动会导致点云虚检。

3.3 查看 Sample 输出

在 RadarDebugTool 工具上选择显示点云、DataCube 信息或调试信息。在绘图窗口点击右键，并选择希望展示的数据，在图 3-1 中分别展示了三个 Sample 在典型应用场景下的点云效果，左侧两个为点云的二维“X-Y”视图，右侧为点云的三维视图：

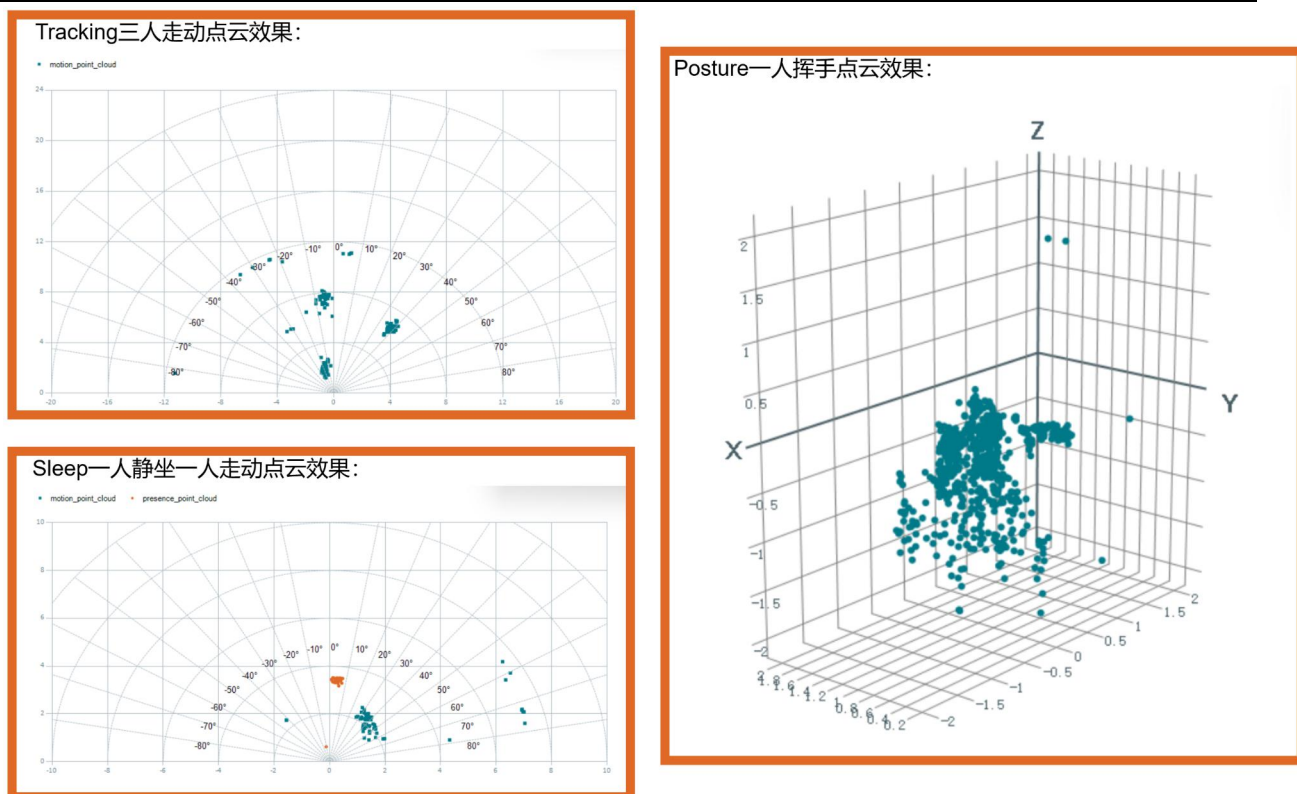


图 3-1 上位机显示点云

图 3-2 中分别展示了“ReportDataCube1D”和“ReportDataCube2D”的 Sample，上报的 DataCube 数据：



图 3-2 上位机展示 DataCube

RadarDebugTool 除查看 Sample 输出外, 还支持修改帧配置, 更多的 RadarDebugTool 工具的使用方法请参考《RS6x_7x_RadarDebugTool_使用手册》。

4 使用 radar framework 进行二次开发

所有 radar framework 支持的特性均可通过静态配置实现。

开发者可选择与需求最为接近的工程，修改配置后编译生成满足应用需求的固件。

也可通过注册回调函数，获取 radar framework 的计算结果，进行后级处理，并增加多种自定义的配置，实现更为丰富的可量产固件。

4.1 代码结构

radar framework 代码结构和关键头文件如下表所示：

```
<home>/Software_Kit/02_SDK/
project/mmwave
|__ common
|__ radar_framework
|__ inc
|__ radar_framework.h          /* radar framework APIs */
|__ radar_framework_default_config.h /* 默认功能配置与堆配置 */
|__ radar_framework_1D_frame_config.h /* 1D 帧结构功能配置 */
|__ radar_framework_2D_frame_config.h /* 2D 帧结构功能配置 */
|__ radar_framework_hif_config.h  /* HIF 配置 */
|__ radar_framework_Interval0_report_config.h /* Interval_0 上报配置 */
|__ radar_framework_motion_pointcloud_config.h /* 移动点云配置 */
|__ radar_framework_presence_pointcloud_config.h /* 存在点云配置 */
|__ radar_framework_soc_config.h  /* SOC 接口配置 */
|__ src
|__ radar_framework.c             /* radar framework 源代码 */
|__ radar_framework_hif_handler.c /* HIF 配置回调 */
|__ radar_framework_report.c      /* 基于 HIF 的数据上报 */
|__ radar_signal_process_1d.c     /* 1D 帧结构回调处理流程 */
|__ radar_signal_process_2d.c     /* 2D 帧结构回调处理流程 */
|__ mmwave                       /* 上报 DataCube 示例工程 */
|__ mmwave_application            /* 点云应用示例工程 */
<home>/Software_Kit/02_SDK/
|__ subsys
|__ hif
|__ hif.h                        /* HIF 主要头文件 */
|__ mmw
|__ mmw_algorithm                /* 点云处理单元模块 */
|__ mmw_application
```

__ mmw_app_pointcloud_config.h	/* 点云计算子系统配置头文件 */
__ mmw_app_micro_pointcloud.c	/* 存在点云实现源文件 */
__ mmw_app_pointcloud.c	/* 移动点云实现源文件 */

4.2 配置 radar framework

radar framework 支持功能性配置和性能配置。功能性配置通过在用户工程中的 prj_config.h 覆盖 project/common/radar_framework/inc/radar_framework_default_config.h 的宏定义使能或失能模块，性能配置通过用户工程中的 prj_config.h 覆盖 subsys/mmw_app_pointcloud_config.h 中的默认值宏实现。

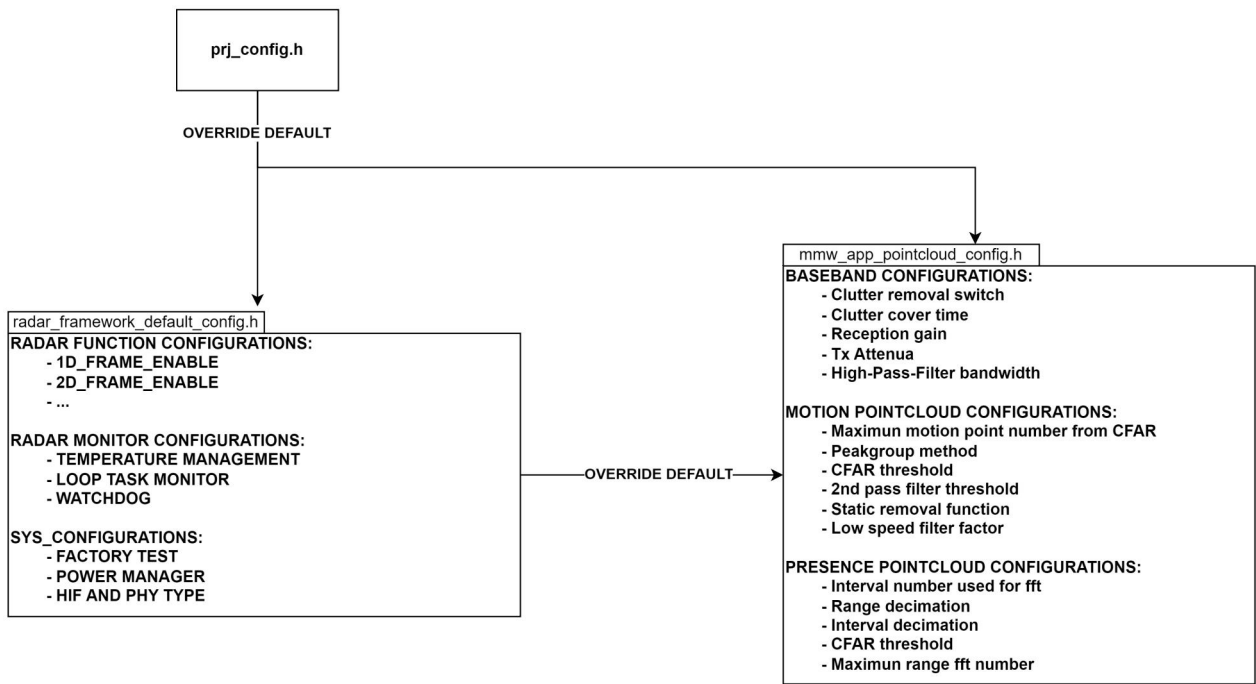


图 4-1 配置参数覆盖逻辑

本章节将介绍'project/common/radar_framework/inc/radar_framework_default_config.h'中的功能配置，关于点云性能的配置位于 subsys/mmw_app_pointcloud_config.h 中，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》。

4.2.1 功能配置指南

功能默认配置模板位于 project/common/radar_framework/inc。radar framework 按下列 8 部分进行功能配置：

1. radar framework 框架全局配置——radar_framework_default_config.h；
2. 1D 帧结构功能配置——radar_framework_1D_frame_config.h；
3. 2D 帧结构功能配置——radar_framework_2D_frame_config.h；
4. 获取 Interval0 距离 FFT 功能配置——radar_framework_Interval0_report_config.h；
5. 移动点云配置——radar_framework_motion_pointcloud_config.h；
6. 存在点云配置——radar_framework_presence_pointcloud_config.h；

7. HIF 配置——radar_framework_hif_config.h;
8. 系统接口配置——radar_framework_soc_config.h。

4.2.2 radar framework 框架全局配置

4.2.2.1 全局使能1D帧结构

使用宏 **CONFIG_RADAR_FRAMEWORK_1D_FRAME_EN** 配置 1D 帧结构的全局使能, 仅在使能此宏时, 波形才能够被配置为 1D 帧模式, 并且所有 1D 帧结构相关代码被编译。

若不使用 1D 帧结构, 建议关闭以节约 SRAM 空间。若此功能使能, 且波形配置为 1D 帧结构时, 可配置缓冲多帧, 由 HOST 一次全部读取, 这在 HOST 读取延时不确定的情况下非常有用。

4.2.2.2 全局使能2D帧结构

使用宏 **CONFIG_RADAR_FRAMEWORK_2D_FRAME_EN** 配置 2D 帧结构功能的全局使能, 仅在使能此宏时, 波形才能够被配置为 2D 帧模式, 并且本章介绍的所有 2D 帧结构相关功能才会被编译。若不使用 2D 帧结构, 建议关闭以节约 SRAM 空间。

4.2.2.3 配置最大支持的距离FFT点数

使用宏 '**CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN**' 配置 radar framework 最大支持的距离 FFT 点数。由于在静态编译时需要依据距离 FFT 的点数确认开辟的最大堆空间, 因此需要在 radar framework 的全局配置中配置 radar framework 支持的最大距离 FFT 点数。

在 radar framework 的初始化阶段会检查实际配置的距离 FFT 点数与最大支持的距离 FFT 点数, 若实际配置的距离 FFT 点数更大, 则会提示错误信息并返回初始化失败。

默认值为: 256。

4.2.2.4 配置DataCube缓冲区

为了解决非实时操作系统中上位机不能及时响应中断, 从而导致在时间裕量很小的时候上报 DataCube 数据丢失的问题, radar framework 设计了多帧 DataCube 的缓冲区上报机制。可以确保上位机还未响应当前帧的 DataCube 中断, 新的一帧已经到来时, 雷达可以缓存若干帧的数据, 当上位机空闲时可以一并并雷达缓存的 DataCube 读走, 从而保证数据是连贯的。

使用宏 **CONFIG_RADAR_FRAMEWORK_DATACUBE_REPORT_SIZE** 和 **CONFIG_RADAR_FRAMEWORK_DATACUBE_LATENCY_REPORT_FRAME_CNT** 分别配置可能支持的单帧最大 DataCube 的大小(Bytes)和需要缓存 DataCube 帧数。以便让 HIF 接口合理分配需要开辟的堆空间。

建议在 SRAM 允许的情况下, 尽可能多地缓存帧数, 以降低丢包率。若想了解 DataCube 缓冲机制的细节, 请参考《RS6x_7x_HIF_参考手册》。

4.2.2.5 配置自适应增益

自适应增益有助于在整个频谱中同时出现 RCS 极大和极小的目标时, 保证它们的回波不溢出或不被量化噪声淹没。

在 2D 帧结构下，可配置硬件在执行距离 FFT 和多普勒 FFT 时开启自适应增益功能。1D 帧结构不支持自适应功能，推荐在上报 DataCube 的应用中不要开启自适应增益功能，会降低传输效率，并增加上位机处理压力。若希望了解更多关于自适应增益的信息，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》。

默认配置：关闭。

4.2.2.6 上报帧信息

使用宏 **CONFIG_RADAR_FRAMEWORK_REPORT_FRAME_INFO** 控制在每次发波前，通过 PSIC 调试协议 (MsgId = 0xC6) 上报帧参数、堆占用情况、计算时间等信息。推荐在需要上报 DataCube 的工程中关闭此宏，以提升上报效率。PSIC 调试协议可在 RadarDebugTool 工具进行可视化。

默认配置：开启。

4.2.2.7 使能心跳包上报

通过宏 **CONFIG_RADAR_FRAMEWORK_REPORT_HEART_BEAT** 控制是否上报心跳包。由于心跳包通常用于开启休眠唤醒的场景，因此默认仅有 Tracking 工程开启此宏上报心跳包，心跳包的默认周期为 10min。若希望了解更多心跳包的详细实现信息，请参考《RS6x_7x_毫米波雷达开发_快速入门》。

4.2.2.8 使能看门狗

通过宏 **CONFIG_MMW_WATCHDOG** 控制开启看门狗，在开启看门狗的情况下，使用宏 **CONFIG_WDG_TIMEOUT_MS** 控制看门狗复位时间，默认时间为 5 秒。

默认所有工程均使能看门狗，若希望了解更多看门狗的详细实现信息，请参考《RS6x_7x_毫米波雷达开发_快速入门》。

4.2.2.9 使能Task Monitor

通过宏 **CONFIG_MMW_LOOP_TASK_MONITOR** 控制开启回调监控，此功能在不启用休眠唤醒的情况下与看门狗功能重叠，但开启休眠唤醒后，在某些休眠等级下，看门狗在系统休眠后不再继续定时，因此使用 Task Monitor 作为辅助监控手段是非常必要的。

默认所有工程均使能 Task Monitor，若希望了解更多 Task Monitor 的详细实现信息，请参考《RS6x_7x_毫米波雷达开发_快速入门》。

4.2.3 1D 帧结构子功能配置

1D 帧结构支持如下配置：

- 配置输出 ADC 数据；
- 上报 1D-DataCube。

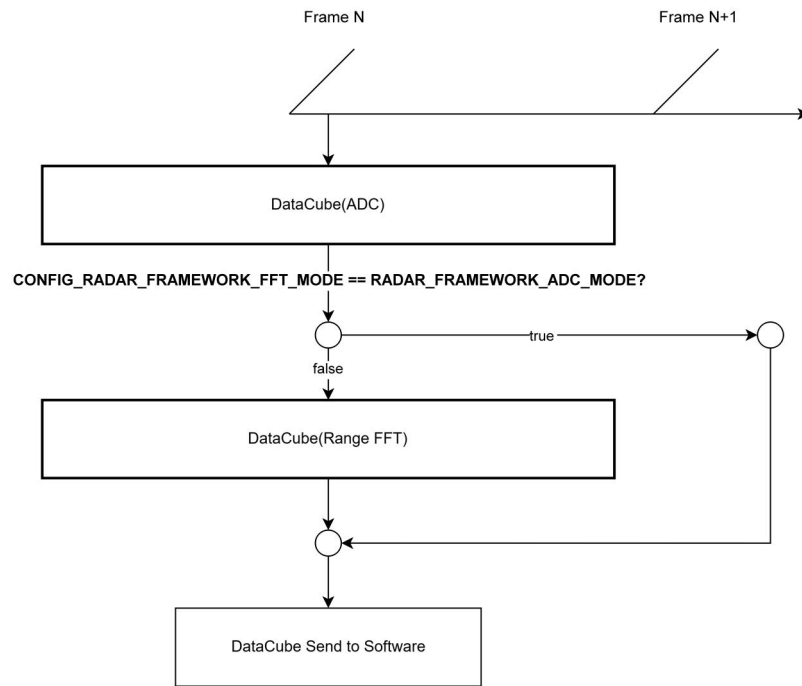


图 4-2 1D 帧结构下的数据通路

4.2.3.1 上报1D-DataCube

使用宏 **CONFIG_RADAR_FRAMEWORK_REPORT_1D_DATA_CUBE** 控制上报 1D-DataCube。

默认值：1（使能上报 1D-DataCube）

4.2.3.2 占用堆配置

使用宏 **HEAP_SIZE_1D_FRAME_CUBE** 指示进行 1D-DataCube 上报功能需要占用的 SRAM 大小。上报 1D-DataCube 所需要的 SRAM 由 HIF 自动计算。

若没有打开上报 1D-DataCube 功能，radar framework 将仅缓存 DataCube 并将其传入用户回调中，占用 SRAM 计算方法如下：

$$\text{距离 FFT 点数} * \text{Rx 天线数} * \text{Tx 天线数} * \text{缓冲帧数} * 4\text{Bytes}$$

4.2.4 2D 帧结构功能配置

4.2.4.1 2D帧结构全局使能

使用 **CONFIG_RADAR_FRAMEWORK_2D_FRAME_EN** 配置 2D 帧结构功能的总开关，仅在使能此宏时，波形才能够被配置为 2D 帧模式、本章介绍的所有 2D 帧结构相关功能才会被编译。若不使用 2D 帧结构，建议关闭该宏以节约 SRAM 空间。默认值：1。

2D 帧流程中，执行 1D 和 2DFFT 均由硬件完成，此过程中支持的流程控制如下图所示：

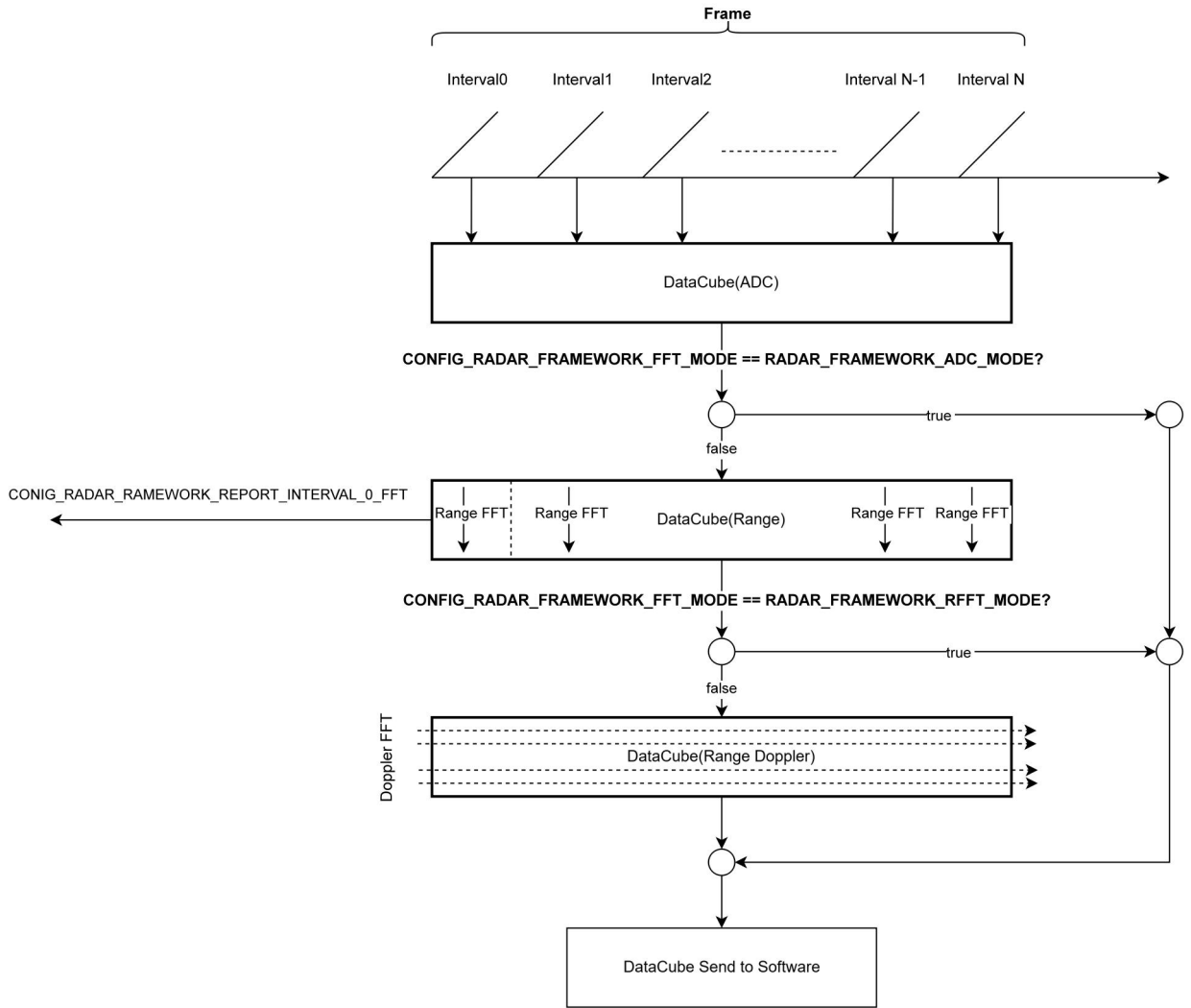


图 4-3 2D 帧结构下的数据通路

4.2.4.2 上报2D-DataCube

使用宏 **CONFIG_RADAR_FRAMEWORK_REPORT_2D_DATA_CUBE** 配置在 2D 帧回调函数中非阻塞地上报 HIF 消息 ID 为 0xC2 的 2D-DataCube。使用单线 SPI 上报 256K 的 2D-DataCube 数据时长典型值为 85ms，不推荐使用 UART 上报 2D-DataCube，关于更多 HIF 协议上报的限制和实现细节，请参考《RS6x_7x_HIF_参考手册》。

默认值：0。

4.2.5 获取 Interval0 的距离 FFT

使用宏 **CONFIG_RADAR_FRAMEWORK_INTERVAL0_RFFT_EN** 控制是否需要缓存 2D 帧模式下的第一个 Interval 的距离 FFT 数据功能（INTERVAL0_RFFT）。INTERVAL0_RFFT 一般用于慢时间维度的呼吸心跳测量。

默认值：0。

使能上报 Interval0 的距离 FFT 数据功能后，还可进一步配置下列宏定义。

- 配置是否上报 INTERVAL0_RFFT

使能 **CONFIG_RADAR_FRAMEWORK_REPORT_INTERVAL0_RFFT** 宏，可在每个回调中非阻塞地上报 HIF 消息 ID 为 0xC2 的 Interval0 的距离 FFT 数据。

- **配置 INTERVAL0_RFFT 占用堆空间**

使用宏 **HEAP_SIZE_1D_CUBE** 配置缓存 INTERVAL0_RFFT 功能需要使用额外的 SRAM。雷达对距离 FFT 数据进行缓冲，缓冲 Buffer 为乒乓结构，所需要的 SRAM

计算方法为：

$$\text{距离 FFT 点数} * \text{Rx 天线数} * \text{Tx 天线数} * 4\text{Bytes} * 2$$

4.2.6 移动点云

使能宏 **CONFIG_MMW_MOTION_POINT_CLOUD** 将开启移动点云计算功能，并在全局堆空间中为移动点云计算预留堆空间。radar framework 在每一次的 2D 帧回调函数中，调用点云计算子系统

(subsys/mmw/mmw_application) 模块中提供的接口，计算点云信息。若希望具体了解软件是如何计算目标位置的，或希望使用 mmw_ctrl API 开发自己的点云算法，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》。

若使能移动点云计算功能，还可配置下列的宏定义进行进一步的功能配置。

4.2.6.1 配置软件CFAR功能

使能宏 **CONFIG_MMW_SW_CFAR_ENABLE** 后，radar framework 将不使用硬件 CFAR 的结果，使用软件 CFAR 进行点云的计算，以提供更快的计算速度但更低密度的点云。这个宏将被 subsys/mmw/mmw_application 中的模块使用，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》。

默认值：0。

注意：若使能了软件 CFAR 功能，自适应增益功能必须关闭，否则会在预编译阶段报错。

4.2.6.2 配置上报移动点云

使能宏 **CONFIG_RADAR_FRAMEWORK_MOTION_POINT_CLOUD_REPORT** 后，radar framework 将使用 HIF 协议上报移动点云。若开发者希望直接使用移动点云进行后续处理时，可关闭此宏。

默认值：1。

4.2.6.3 配置移动点云最大CFAR点数

使用宏 **CONFIG_MMW_MAX_MOTION_CFAR_NUM** 配置移动点云的最大 CFAR 点数。降低 CFAR 点数可缩短计算时间，当实际探测到的移动点云数超过配置值后，会舍弃掉远距离的目标。

默认值：512

4.2.6.4 移动点云堆配置

使用宏 **HEAP_SIZE_MOTION** 配置移动点云堆大小，移动点云所需要的堆大小与配置的最大 CFAR 点数有关。移动点云所使用的堆大小为：

移动点云 CFAR 结果存储空间： $512 * 4(\text{Bytes}) + \text{CONFIG_MMW_MAX_MOTION_CFAR_NUM} * 4(\text{Bytes})$ ，其中硬件 CFAR 结果至多 512 个，软件 CFAR 结果数与最大 CFAR 点数相同；

移动点云计算结果存储空间： $\text{CONFIG_MMW_MAX_MOTION_CFAR_NUM} * 24(\text{Bytes})$ ；
移动点云上报存储空间： $\text{CONFIG_MMW_MAX_MOTION_CFAR_NUM} * 12(\text{Bytes})$ ，需配置移动点云上报；
移动点云初始化使用空间： $\text{CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN} * 8$ ，
移动点云总占用堆大小：移动点云 CFAR 结果存储空间 + 移动点云计算结果存储空间 + 移动点云上报存储空间 + $\text{CONFIG_MMW_MAX_MOTION_CFAR_NUM} * 1(\text{Bytes})$ （低速过滤所需堆大小）+ 移动点云初始化使用空间。

4.2.7 存在点云

使能宏 **CONFIG_MMW_PRESENCE_POINT_CLOUD** 将开启存在点云计算功能，并在全局堆空间中为存在点云计算预留堆空间。radar framework 在每一次的 2D 帧回调函数中，调用 subsys/mmwr/mmwr_application 模块中提供的接口，计算存在点云信息。

存在点云的基本计算流程如下：根据存在点云配置的分频数 N，每 N 帧缓冲 DataCube 中的 0 号速度 bin，组成慢时间维度的 DataCube，在慢时间维度上执行多普勒 FFT，并使用 CFAR 进行存在目标检测。

若使能存在点云功能，还可进一步配置下列宏：

4.2.7.1 配置存在点云最大点数

使用宏 **CONFIG_MMW_PRESENCE_POINT_MAX** 配置存在点云的最大检测点数，降低最大检测点数可缩短计算时间，但可能舍弃掉远距离的目标。

默认值：256。

4.2.7.2 配置存在点云CFAR模式

使用宏 **CONFIG_MMW_MICRO_POINT_CLOUD_CFAR_MODE** 配置存在点云的 CFAR 检测算法。传统 CFAR 使用双边底噪进行自适应的底噪估计，为了节约 SRAM 和计算量，存在点云算法的 CFAR 支持仅使用单边底噪进行 CFAR 底噪估计，在空旷环境下性能几乎不会受到损失，但可节约 10K SRAM 空间。

将 **CONFIG_MMW_MICRO_POINT_CLOUD_CFAR_MODE** 定义为 0，在存在点云中使用单边 CFAR 模式，定义为 1 使用双边 CFAR 模式。

默认值：0（单边 CFAR 模式）。

4.2.7.3 配置存在点云的距离Bin个数

使用宏 **CONFIG_MMW_PRESENCE_RANGE_BIN_NUM** 配置存在点云的最远检测距离门序号。存在点云探测能力有限，且探测范围越大，环境中干扰也会更大，因此一般不需要检测到过远的距离。设置此宏的目的在于根据应用需求限制存在点云的探测范围，可大幅减少存在点云所需要的堆空间和计算时间。

默认值：256（存在点云仅检测前 256 个 bin）。

4.2.7.4 配置存在点云距离抽取系数

使用 **CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC** 配置存在点云的距离抽取率。此宏默认为 2，即存在点云仅计算距离 bin 为 0/2/4/6/8...的目标，可减少计算量，并且减少一半的慢时间维度 DataCube 的存储空间，但会减少目标存在点云的密度。

默认值：2（距离抽取率为 2）。

4.2.7.5 配置存在点云缓冲Interval数

使用宏 **CONFIG_MMW_PRESENCE_INTERVAL_NUM_LOG2** 配置存在点云缓冲的 Interval 数，开发者在定义此宏时需配置为 LOG 值，这些 Interval 将会被用于执行慢时间维度的 FFT 以获得存在点云谱。

默认值：3（Interval 缓存数为 8）。

4.2.7.6 配置存在点云多普勒FFT点数

使用宏 **CONFIG_MMW_PRESENCE_DFFT_LEN_LOG2** 配置存在点云对慢时间维度的 DataCube 进行 FFT 时的长度，开发者在定义此宏时需配置为 LOG 值，需要注意的时：配置的多普勒 FFT 点数必须大于等于缓冲的 Interval 数。

默认值：4（多普勒 FFT 长度为 16）。

4.2.7.7 存在点云堆配置

1. 使用宏 **HEAP_SIZE_PRESENCE** 配置存在点云堆大小，存在点云所需要的堆大小与存在点云 CFAR 模式和抽取系数相关。存在点云所使用的堆大小为：
2. 存在点云计算结果存储空间：**CONFIG_MMW_PRESENCE_POINT_MAX * 24(Bytes)**；
3. 计算缓存 Interval 和多普勒 FFT 数： $1 << \text{CONFIG_MMW_PRESENCE_INTERVAL_NUM_LOG2}$ 和 $1 << \text{CONFIG_MMW_PRESENCE_DFFT_LEN_LOG2}$ ；
4. 存在点云实际使用距离 bin 个数： $\text{CONFIG_MMW_PRESENCE_RANGE_BIN_NUM} / \text{CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC} + 1$ ；
存在点云 Cube 大小：存在点云实际使用距离 bin 个数 * Rx * Tx * (Interval 数 + 1) * 4(Bytes)；
5. 存在点云 Interval 缓冲空间： $\text{Rx} * \text{Tx} * (\text{Interval 数} + 1) * 4(\text{Bytes})$ ；
6. 计算最大窗长度所需的单元，其长度根据宏 **CONFIG_MMW_MICRO_POINT_CLOUD_CFAR_MODE** 配置为单边/双边 CFAR 模式确定：
 - 单边 CFAR 模式： $(37 / \text{CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC} + 1) * 4(\text{Bytes})$ ；
 - 双边 CFAR 模式： $(73 / \text{CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC} + 1) * 4(\text{Bytes})$ ；
7. 存在点云计算缓存空间：存在点云实际使用距离 bin 个数 * 6(Bytes) + (多普勒 FFT 数 * (Rx * Tx + 1)) * 窗长度空间单元)；
8. 存在点云上报存储空间：**CONFIG_MMW_PRESENCE_POINT_MAX * 12(Bytes)**；
存在点云总占用堆大小：存在点云计算结果存储空间 + 存在点云存储慢时间维度的 DataCube 空间 + 存在点云 Interval 缓冲空间 + 存在点云 FFT 空间 + 存在点云上报存储空间。

4.3 基于 radar framework 的应用配置

SDK 中提供了多种基于 radar framework 的 Sample，本章主要描述根据不同的应用，这些 Sample 的毫米波雷达波形和点云性能的配置逻辑，以及如何对功能和性能进行裁剪。

4.3.1 波形配置

波形配置的相关参数与关键时间的定义如下图所示，更多关于波形配置的详细信息请参考《RS6x_7x_毫米波雷达_参考手册》。

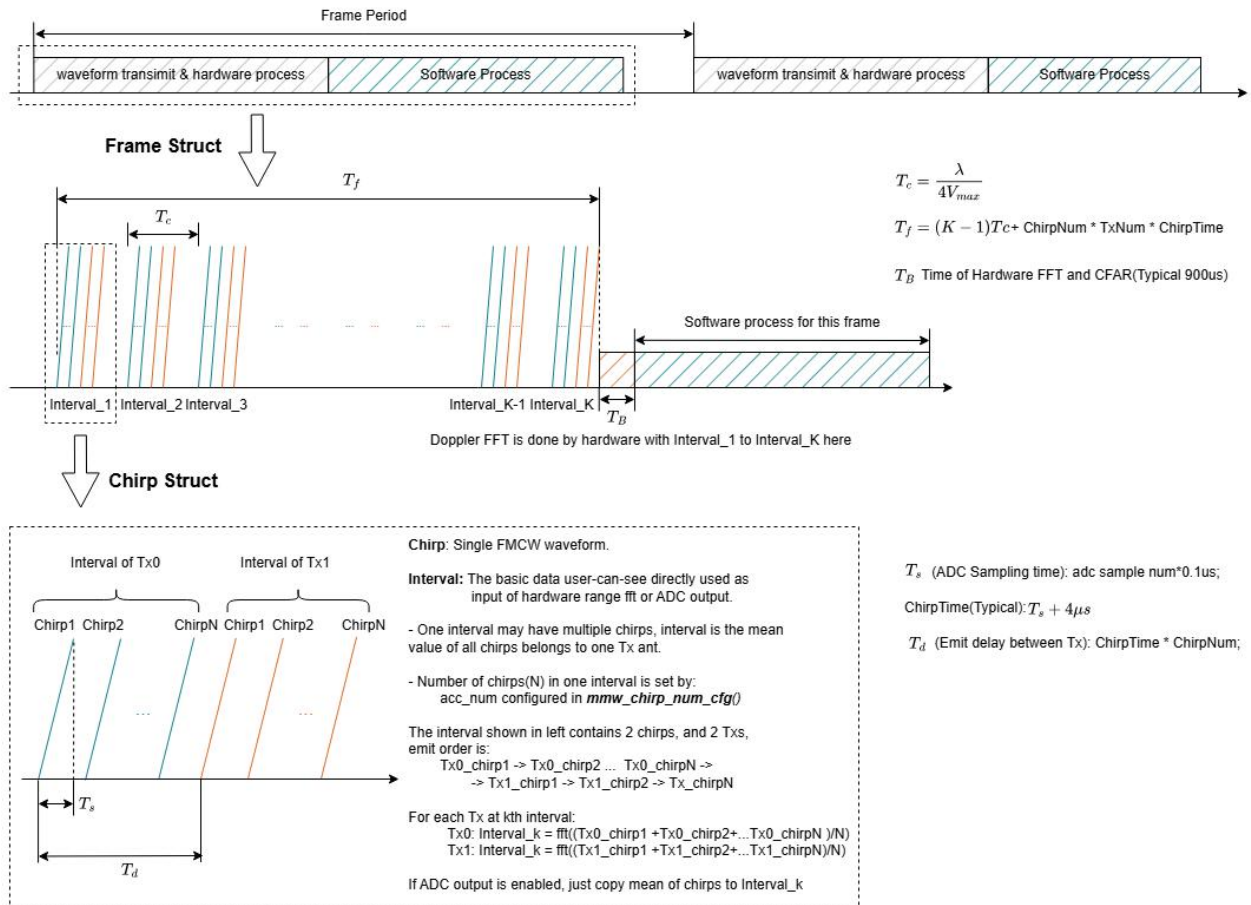


图 4-4 波形结构示意图

波形配置包含下列几个可配置的项次。

4.3.1.1 MIMO模式

使用宏 `RADAR_FRAMEWORK_SAMPLE_MIMO_MODE` 配置天线工作模式，若使用 6130 系列芯片，此值仅可配置为 `MMW_MIMO_1T3R`，若使用 6240 系列芯片，此值仅可配置为 `MMW_MIMO_2T4R`。

4.3.1.2 起始频率

使用宏 `RADAR_FRAMEWORK_SAMPLE_START_FREQ_MHZ` 配置起始频率。其中 RS6240 系列芯片支持的全频率范围为 57GHz-66GHz，RS6130 系列芯片支持频率范围为 58GHz-64GHz。

4.3.1.3 帧类型

使用宏 `RADAR_FRAMEWORK_SAMPLE_FRAME_TYPE` 配置默认需要发送的帧结构。仅在使能宏 `CONFIG_RADAR_FRAMEWORK_1D_FRAME_EN` 或 `CONFIG_RADAR_FRAMEWORK_2D_FRAME_EN` 后可配置对应的帧结构，否则会在帧配置时报错误信息。若配置为 1D 帧结构时，图 4-4 中的 Interval_K 恒为 1，且数据流程按照图 4-2 执行。

4.3.1.4 距离FFT点数(log2)

使用宏 **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** 配置以 log2 的形式的距离 FFT 点数，最大值为 10 (不开启自适应增益) 或 9 (开启自适应增益)。例如，若 **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** 配置为 8，那么距离 FFT 点数和 ADC 采样点数为： $2^8=256$ 。

配置说明 1： 一般推荐将距离 FFT 点数(log2)配置为 7/8/9，可获得较好的性能与功耗的平衡。radar framework 仅可支持静态配置距离 FFT 为 2 的幂次，这适用于大部分场景。过大的距离 FFT 点数会大幅增加计算压力和存储压力，过小的距离 FFT 点数会降低探测能力。

配置说明 2： 距离 FFT 点数需要小于宏 **CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN** 所表示的距离 FFT 点数；

配置说明 3： 若 **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** 配置为 **UINT16_MAX**，则 radar framework 不会使用 **range_fft_len_log2** 的值作为 FFT 点数，而会通过结构体 **RadarConfig_t** 中的元素 **max_range_mm / range_resolution_mm** 计算 ADC 采样点数，并选择不小于 ADC 采样点数的 2 的幂次作为距离 FFT 点数，即此时距离 FFT 点数为：

$$\text{power}(2, \text{ceil}(\log_2(\text{max_range_mm}/\text{range_resolution_mm})));$$

配置说明 4： 若配置为不执行距离 FFT，那么仅支持配置距离 FFT 点数为 2 的幂次。

4.3.1.5 距离分辨率

使用宏 **RADAR_FRAMEWORK_SAMPLE_RANGE_RESOLUTION_MM** 配置距离分辨率。较小的距离分辨率可产生较高密度的点云，和更精准的测距。距离分辨率和距离 FFT 点数共同决定了最远探测距离：

最远探测距离 = $(1 < \text{RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2})$
 $\times \text{RADAR_FRAMEWORK_SAMPLE_RANGE_RESOLUTION_MM};$

4.3.1.6 多普勒FFT点数(log2)

使用宏 **RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2** 配置以 log2 的形式的多普勒 FFT 点数，最大值为 10。例如，若 **RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2** 配置为 8，那么多普勒 FFT 点数和 Interval 数为： $2^8=256$ 。

注意： DataCube 最大为 256KB(RS624x)或 192KB (RS6130)，固件内置压缩算法，最多支持 1 倍压缩，因此最大支持的 DataCube 大小为 512K 和 384K。DataCube 占用空间计算方法如下：

$$\text{距离 FFT 点数} \times \text{多普勒 FFT 点数} \times \text{Tx 天线数} \times \text{Rx 天线数} \times 4\text{Bytes(IQ)}$$

为了满足上述要求，要求

$$\text{RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2} + \text{RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2} \leq 14;$$

4.3.1.7 速度分辨率

使用宏 **RADAR_FRAMEWORK_SAMPLE_VEL_RESOLUTION_MM** 配置速度分辨率。通过配置速度分辨率和多普勒 FFT 点数可以决定雷达的有效速度测量范围为：

$$[-\text{速度分辨率} \times \text{多普勒 FFT 点数}/2, \text{速度分辨率} \times \text{多普勒 FFT 点数}/2]$$

速度分辨率影响了 Frame 发射波形的长度。可按下列公式快速估算 Frame 的发波时间

$$T_f = \frac{\lambda}{2V_{res}}$$

4.3.1.8 Chirp累积数(log2)

使用宏 `RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2` 配置一个 Interval 内的 Chirp 累积数。雷达会将一个 Interval 内的所有 Chirp 在复数域进行平均后，作为此 Interval 的采样，进行距离 FFT。一个 Interval 内 Chirp 的个数为： $(1 \ll \text{RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2})$ 。使用 Chirp 累积数可提升目标探测能力，但需要注意的是，如果无模糊速度太小，可能导致两个 Interval 间无法放下所有的 Chirp，从而导致雷达发波异常。

推荐 `RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2` 设置为小于等于 3，其中设置为 0 时，表示一个 Interval 仅发送一个 Chirp。

4.3.1.9 FFT模式

使用宏 `CONFIG_RADAR_FRAMEWORK_FFT_MODE` 配置为 `RADAR_FRAMEWORK_ADC_MODE`/`RADAR_FRAMEWORK_RFFT_MODE`/`RADAR_FRAMEWORK_2DFFT_MODE` 以分别配置图 4-3 中的不执行距离 FFT、仅执行距离 FFT 或执行完整的 2D-FFT 流程。

若将 `CONFIG_RADAR_FRAMEWORK_FFT_MODE` 配置为 `RADAR_FRAMEWORK_ADC_MODE`，雷达不再进行距离 FFT 和多普勒 FFT，上报的 DataCube 中的数据被替换为每个 Interval 的 ADC 采样值。

- **使用限制 1：**配置为 `RADAR_FRAMEWORK_ADC_MODE` 时，移动和存在点云无法计算，并会在 app 初始化时打印提示信息；
- **使用限制 2：**配置为 `RADAR_FRAMEWORK_ADC_MODE` 时，**最大探测距离必须为距离分辨率的 2 的幂次方倍、最大测量速度必须为速度分辨率的 2 的幂次方倍**，否则可能发生数据错乱。

若将 `CONFIG_RADAR_FRAMEWORK_FFT_MODE` 配置为 `RADAR_FRAMEWORK_RFFT_MODE`，可控制雷达不执行多普勒 FFT，此时，上报的 DataCube 中的数据为 2D 帧中每个 Interval 的距离 FFT 数据。

- **使用限制 1：**配置为 `RADAR_FRAMEWORK_RFFT_MODE` 时，移动和存在点云无法计算，并会在 app 初始化时打印提示信息；
- **使用限制 2：**配置为 `RADAR_FRAMEWORK_RFFT_MODE` 时，**最大测量速度必须为速度分辨率的 2 的幂次方倍**，否则可能发生数据错乱。

若将 `CONFIG_RADAR_FRAMEWORK_FFT_MODE` 配置为 `RADAR_FRAMEWORK_2DFFT_MODE`，对于 2D 帧结构来说将执行距离 FFT 和多普勒 FFT，对于 1D 帧结构来说仍然只执行距离 FFT。

4.3.1.10 静态杂波收敛模式

使用宏修改 `RADAR_FRAMEWORK_SAMPLE_CLUTTER_HALT_METHOD` 可配置静态杂波的收敛模式（仅开启静态杂波移除功能有效）。静态杂波抑制的基本工作流程是雷达硬件对环境的静止目标回波（称为静态杂波）进行估计，并在每一个 Interval 执行距离 FFT 之前将其减掉。在 DataCube 中，这些静止目标能量会被得到抑制，从而静止目标不会被 CFAR 检测到。

静态杂波支持：

- 持续更新；
- 收敛后不更新；
- 自动更新

三种更新模式。

- 持续更新：雷达对静态杂波持续进行估计跟踪。
这是当环境可能发生变化时常用的一种更新方法。优点在于可以持续跟踪环境中静止目标的能量，当环境中有新出现的目标、或原有目标消失时，不会造成持续的虚检。缺点在于，如果感兴趣目标处于静止状态一段时间后，感兴趣目标会被作为环境目标进行抑制，导致无法测量；
- 收敛后不更新：雷达在首次静态杂波收敛后，不再对静态杂波进行估计。若开启了静态杂波移除功能，每次更新都将减去这个固定的静态杂波。
此方法优点在于任何和初始环境的差异都可以被检测出来，但若雷达发生了位置移动，或环境中物体的变化不是期望被探测到的，会产生持续的虚检。此外环境温度若发生了变化，也将导致首次收敛的静态杂波不再准确，因此若使用收敛后不更新的配置，需要在应用层定时主动进行静态杂波的更新，以确保稳定性。
- 自动更新：仅在开启移动点云功能时有效。
当帧移动点云没有检测到点云时，为下一帧开启静态杂波更新，否则暂停静态杂波更新。

注意：由于该检测机制以当帧是否存在点云，来控制下一帧的静态杂波更新，因此开发者不可将点云计算子模块中的静态杂波更新时间设置得过小，推荐设置为 4 及以上。另外，若雷达信噪比设置若过低，在环境中发生持续虚检，也不会进行静态杂波的更新。

4.3.2 点云性能配置指南

性能配置主要包含波形配置和点云计算子模块配置。
波形配置由 radar framework 直接使用 prj_config.h 中相关的宏进行控制，性能配置通过在 prj_config.h 中覆盖 subsys/mmwave_application/mmwave_pointcloud_config.h 中的宏进行配置。
关于点云计算子模块的更多配置，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》。

4.3.3 功能裁剪指南

开发者在使用雷达 sample 开发过程中，可根据自身需求，将不需要的功能关闭，减小 SRAM 占用，从而为开发者自己的应用留出更多的 SRAM 空间。
除此之外，还有以下方法可以进行功能裁剪。

1. 若开发者使用了雷达 sample 中提供的移动点云过滤方法，在 SRAM 空间较为紧张时，可以在保证能检测到有效目标的情况下，将移动点云最大 CFAR 点数 **CONFIG_MMW_MAX_MOTION_CFAR_NUM** 减小；
2. 若开发者使用了存在点云功能，在 SRAM 空间较为紧张时，可以将存在点云最大 CFAR 点数减小，或在保证性能的同时配置较大的距离抽取率，具体配置方法参考章节 4.2.7。

4.3.4 典型波形配置

以 6240 系列芯片为例，五个雷达 Sample 的典型波形配置如下表所示：

工程名	ReportDataCu be1D	ReportDataCu be2D	Posture	Sleep	Tracking
-----	----------------------	----------------------	---------	-------	----------

工程名	ReportDataCu be1D	ReportDataCu be2D	Posture	Sleep	Tracking
MIMO 模式	MMW_MIMO_ 2T4R	MMW_MIMO_ 2T4R	MMW_MIMO_ 2T4R	MMW_MIMO_ 2T4R	MMW_MIMO_ 2T4R
起始频率	57GHz	57GHz	57GHz	58.88GHz	58.88GHz
帧类型	1DFFT	2DFFT	2DFFT	2DFFT	2DFFT
距离 FFT 点数	256(2^8)	256(2^8)	256(2^8)	256(2^8)	256(2^8)
距离分辨率	0.08m	0.08m	0.025m	0.04m	0.08m
多普勒 FFT 点 数	32(2^5)	32(2^5)	32(2^5)	32(2^5)	32(2^5)
速度分辨率	0.25m/s	0.25m/s	0.1m/s	0.2m/s	0.2m/s
Chirp 累积数	1(2^0)	1(2^0)	1(2^0)	1(2^0)	4(2^2)
静态杂波收敛 模式	不使能	不使能	不使能	不使能	持续更新

4.3.5 典型性能指标

下表展示了各关键模块的单次执行时间、512 个有效移动点云的计算时间以及单帧存在点云的计算时间：

表 4-1 典型计算时间

512 有效移动点云计算时间	21.3ms
存在点云计算时间	6~7ms/帧

各 Sample 剩余 SRAM 大小如下表所示（cdk_6240_cpuf）：

表 4-2 可用 SRAM 大小

工程名	剩余 SRAM 大小\KB
Posture	89
Sleep	74
Tracking	150

图 4-5 2D 数字信号处理框架

4.5 HIF 配置框架介绍

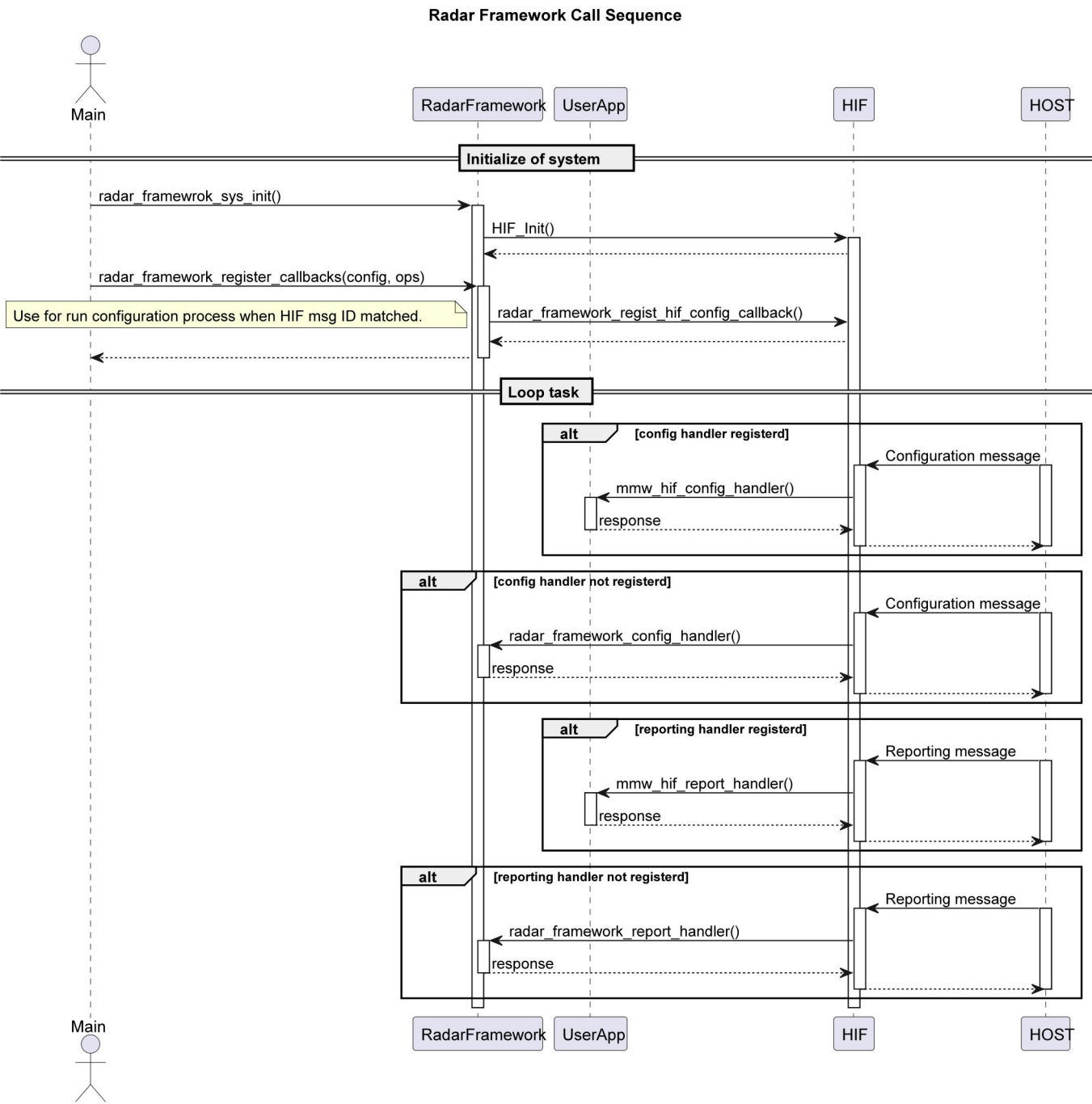


图 4-6 HIF 配置框架调用流程

5 FAQ

5.1 如何提升点云密度？

点云密度由 CFAR 阈值、距离分辨率、距离 FFT 点数、速度分辨率、多普勒 FFT 点数共同决定。其中，CFAR 阈值、距离分辨率、速度分辨率设置的值越小，点云密度越大，距离 FFT 点数、多普勒 FFT 点数越大，点云密度越大。但点云密度增大后，杂点也会相应增加。

5.2 如何提升可用的 SRAM？

开发者可根据下列途径来压缩代码所需的空间：

- 将 `CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN` 宏设置为可能支持的最大距离 FFT 点数(log2)；
- 在 `prj_config.h` 中关闭不需要的功能，以释放这些功能额外开辟的堆内存；
- 参考 `HEAP_SIZE_PRESENCE` 的计算方法，可通过减少存在点云的距离 bin 个数、距离抽取点数、缓冲的 Interval 个数，从而达到减少 SRAM 占用的目的。

5.3 为何点云输出会卡顿，如何减少计算时间？

当下一帧已经发送完成，并产生了中断，当前帧计算还未完成时，就不会响应下一帧的中断，从而导致下一帧的数据不会被计算，导致点云卡顿。针对点云卡顿的问题，可通过下列方式进行改进：

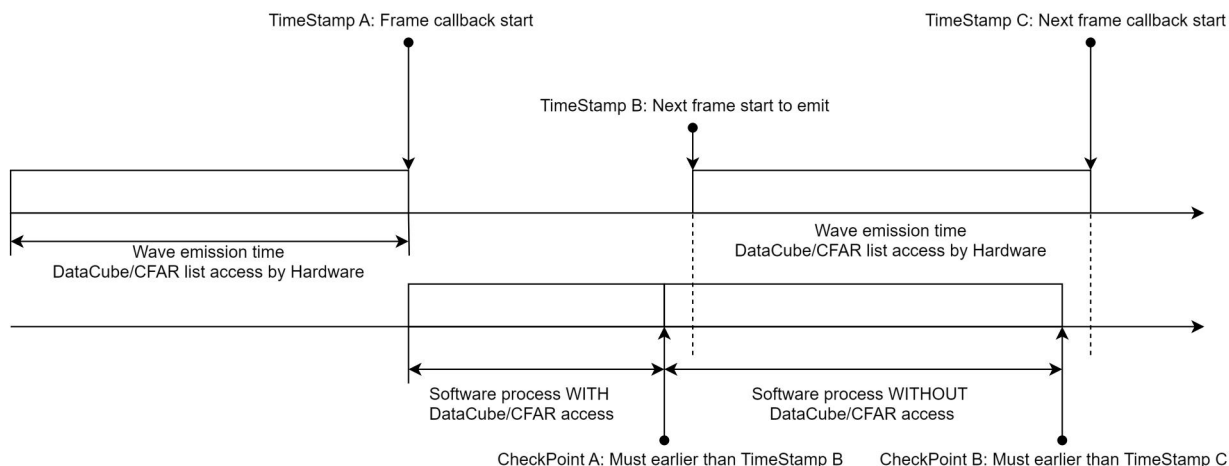
- 对于移动点云应用，可通过修改宏 `MMW_MAX_MOTION_CFAR_NUM` 配置为更小的值，限制移动点云数量上限，从而降低计算时间，在最差情况下，输出一个移动点云，大约需要 64us；
- 可通过增加存在点云的距离抽取率
`CONFIG_RADAR_FRAMEWORK_PRESENCE_POINT_CLOUD_RANGE_DECM`，降低计算微动谱的 FFT 次数，从而达成降低计算时间的需求；
- 缩小宏 `RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2` 或 `RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2`，可降低距离和多普勒 FFT 点数，每降低一倍距离 FFT 点数或速度 FFT 点数，移动点云计算时间大约变为原来的一半，同时存在点云计算时间也会下降。但点云密度也会下降；
- 在点云计算子模块（`subsys/mmw/mmw_application/mmw_app_pointcloud_config.c`）中配置合理的距离过滤范围和速度过滤范围，不计算无用点云，达到减少计算时间的目的。关于点云计算子模块的使用方法，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》；
- 在点云计算子模块中提高信噪比阈值(`threshold_snr_pwr_lin`)。提高信噪比可以降低有效点云数目，消除杂点，从而减少计算时间。关于点云计算子模块的使用方法，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》；

5.4 波形配置有没有最佳实践？

- 距离相关配置
 - 若没有特殊要求或对雷达数字信号处理不熟悉，请尽可能使用 **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** 宏来配置距离 FFT 点数，以确保距离谱计算不存在补零操作；
 - 若不知应该设置多少点距离 FFT，建议设置距离 FFT 点数为 256 或 512，若计算压力过大，优选 128 或 256，否则优选 256 或 512；
 - 若期望最远探测距离为 x ，则建议：
 - ◆ 根据上述原则确认距离 FFT 点数，最远探测距离为距离分辨率*距离 FFT 点数；
 - ◆ 通过调整距离分辨率，使得最远探测距离为 $2x/3 \sim x/0.8$ 之间，可较为完整地利用整个频谱，且不使用最后 20% 处于滤波器过渡带的频率；
- 速度相关配置
 - 确定需要测量的速度上限，一般行人测量最大测量速度 3.2m/s，测量的速度范围是 -3.2m/s~3.2m/s；
 - 建议一般设置多普勒 FFT 点数为 32，若计算压力过大，可选择 16；
 - 根据速度上限和多普勒 FFT 点数计算速度分辨率为： $2 * \text{最大测量速度} / \text{多普勒 FFT 点数}$ ；

5.5 计算时间有什么约束？

radar framework 在两个时间点会检查当帧数据是否满足约束：



- (1) CheckPoint A: 当完成 DataCube 访问时（通常为完成移动点云计算和微动点云缓存），radar framework 会检查下一帧是否已经开始发射。若已经开始发射，则软件和硬件可能已经发生了 DataCube 访问冲突，当前帧数据不可靠，radar framework 将不再进行后续计算，等待下一次回调；
- (2) CheckPoint B: 当帧数据没有完成处理时，下一帧发波已经完成并产生中断，此时下一帧的中断不会被响应，导致下一帧数据不会被处理。

5.6 如何提升雷达视野内对微弱反射目标的探测能力？

- (1) 首先推荐使用 **RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2** 提升 Chirp 累积数来提升对微弱反射目标的探测能力。使用这种方案，可在几乎不修改原有波形参数、点云密度的条件下，仅提升探测能力；
- (2) 其次可通过增大 **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2**、**RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2** 来提升微弱目标的探测能力，但提升这两种配置会对点云密度和算力产生影响，上层算法可能需要进行相应的调整；
- (3) 最后可通过降低 CFAR 阈值，并在应用层对点云进行二次过滤实现对微弱反射目标的探测能力，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》了解如何调整 CFAR 阈值；

RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2 每提升 1，理论上检测性能等价于距离 FFT 点数或多普勒 FFT 点数提升一倍。

5.7 为什么容易在近距离处存在点云有虚检？

雷达具有后向辐射的能量，坐在雷达后方能量较强的目标可能也会被雷达捕捉到。存在点云检测灵敏度较高，可能检测到坐在雷达后方的人，因此要求在测试时，测试人员尽可能远离雷达。若仍然存在虚检，需要在 SDK 中调整存在点云的阈值，请参考《RS6x_7x_毫米波雷达_点云流程_参考手册》。

5.8 为什么移动点云在室内有很多假目标？

在狭小且具有大量金属目标的环境下，雷达可能产生大量的虚假目标，这些虚假目标主要来自于两个方面：

1. 探测到来自多种传播路径的目标，典型的多径传播干扰如下：

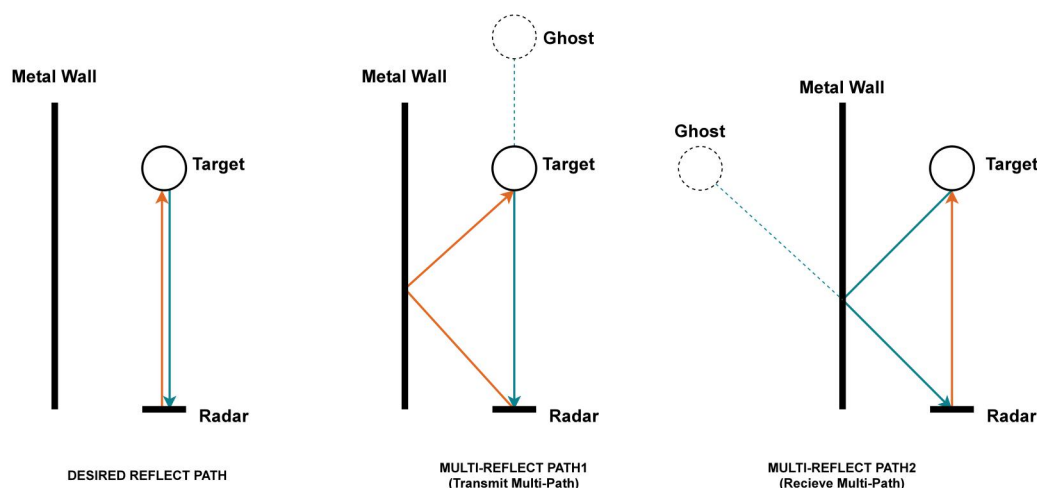


图 5-1 多径干扰

2. 移动目标对身后的静止目标有遮挡，但遮挡关系不是稳定的，静止目标的能量被人的运动频率“幅度调制”，进入速度维度，从而将静止目标识别为运动目标。若将雷达比作光源，那么这些静止目标在雷达眼中作为人的“影子”被检测到。

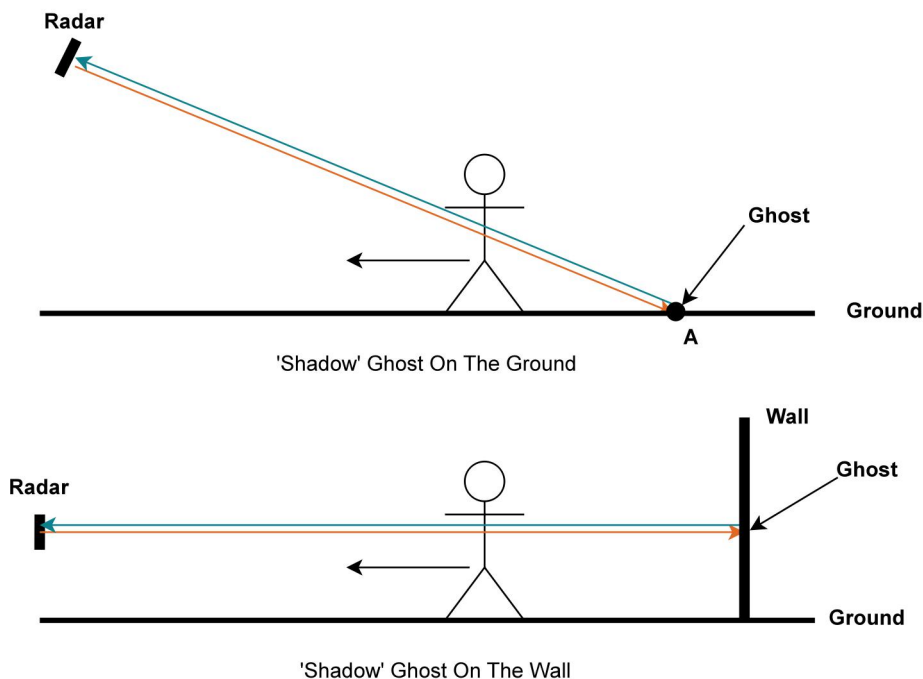


图 5-2 “影子”假目标

开发者需要根据能量、信噪比、运动规律等多方面在后续处理中剔除这些假目标。

5.9 为什么使用 RadarDebugTool 配置可能失败？

SDK 发布包中的 Tracking/Posture/Sleep 工程的固件最大仅支持 256 点距离 FFT，因此若配置距离 FFT 点数超过 256，会提示配置失败。若需要修改最大支持的距离 FFT 点数，请修改 `CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN` 宏为 512，并重新编译相关固件。

5.10 为什么有的 Chirp 累积数下频谱和点云会异常？

ACC 配置过大时，Interval 间无法插入配置的 chirp 数，导致雷达发波异常。但目前 SDK 无法支持此种异常情况检测，因此当观察到这种情况时，可尝试调小 ACC 配置数，观察异常是否消除。

5.11 调试串口打印 malloc fail 怎么办？

由于堆开辟不足导致申请内存失败，此时需要修改 `radar_framework_default_config.h` 中控制堆大小的 `CONFIG_HEAP_SIZE`，增加相应的内存。

5.12 调试串口打印的错误信息有什么含义？

“The HIF task of the previous frame was not completed!”：表明计算时序违反了 5.5 中的 CheckPointB，需要检查 HOST 是否已经及时响应 HIF 的中断，并发送 Poll 指令请求 HIF 数据。

“[RADAR_SIGNAL_2D_PROCESS] Calculate TIMEOUT!”：表明计算时序违反了 5.5 中的 CheckPointA，此时表明点云数量过多，若持续出现此情况，可考虑增长帧周期或减小 CONFIG_MMW_MAX_MOTION_CFAR_NUM 宏的值，以降低计算压力

6 附录

6.1 使用 ADC 和 1D-DataCube 数据进行数字信号处理

本章节给使用 ADC 或者 1D-DataCube 数据，进行点云算法研发的开发者提供基础思路。

6.1.1 速度补偿系数

当目标在进行移动时，由于多发射天线间的发波间隔，会导致不同发射天线间存在相位差，从而造成测角误差，因此在测角前，需要对发射天线进行速度补偿。使用 2D-FFT 数据时，硬件已经进行了速度补偿计算，但使用 ADC 或者 1D-DataCube 数据时，需要开发者进行软件速度补偿。

以 RS624X 为例，对于 Tx1 的角度校准公式可以表示为：

$$\text{deta_phi} = (-1) * 2 * \pi * \text{dop_idx_shift} * \text{deta_phi_coef}$$

其中，dop_idx_shift 为进行 doppler fft shift 后的速度索引，deta_phi_coef 为速度补偿系数，由速度分辨率和 ACC 数决定。

$$\text{ant_vel_calib_data}[4 \sim 7] = \text{ant_data}[4 \sim 7] * (\cos(\text{deta_phi}) + j * \sin(\text{deta_phi}))$$

ant_data 和 ant_vel_calib_data 分别为进行速度补偿前和后的数据，且仅对 Tx1 的数据进行补偿。

开发者可通过 `mmw_get_dec_fcw()` 获得速度补偿系数，并在测角前进行速度补偿。

6.1.2 角度校准值

使用 ADC 或者 1D-DataCube 进行信号处理开发时，需注意是否进行了时域硬件的角度校准，详情请参考 RS6x_7x_毫米波雷达_参考手册。若未进行角度校准的话，需要在测角前进行软件校准，开发者可通过 `mmw_ant_calib_data_get()` 获得角度校准矩阵。

6.1.3 ADC 数据点云计算流程

以 RS624X 系列模组的 ADC 数据进行点云计算为例，其流程如图 6-1 所示，主要分为以下步骤：

1. 获得速度补偿系数 deta_phi_coef 与天线校准数组 ant_calib_arr：
 - (1) deta_phi_coef：为 U32Q20 的值，根据雷达的帧配置决定；
 - (2) ant_calib_arr：长度为 8，S16Q15 的 IQ 值，每个模组的天线校准数组可能不同，且不随雷达配置而改变；
2. 进行 FFT 处理，将 ADC 数据转化为 DataCube，推荐进行 FFT 处理的时候添加汉宁窗，需要注意在进行 FFT 后需要进行 FFT shift；
3. 进行 CFAR 筛选目标，通过 DataCube 转化为目标的距离与速度信息；
4. 通过速度补偿系数进行速度补偿，补偿目标移动导致 tx0 与 tx1 之间的相位差；
5. 通过校准数组进行天线校准，修正由于天线误差导致的测角误差；

6. 将通过速度补偿和天线校准后的数据进行测角，得到目标的角度信息。

关于 CFAR 以及测角的流程可参考 `mmw_cfar_detection_sw_post_process()` 以及 `mmw_process_angle_process()`。

使用限制 1：步骤 4 中速度补偿是对于 tx0 和 tx1 之间的相位差进行补偿，若开发者使用的是 RS613X 系列芯片，则仅有一根发射天线，因此无需进行此步骤；

使用限制 2：如果开发者使用 RS613X 系列芯片，默认进行了硬件校准，此时不再需要进行软件角度校准。

使用限制 3：如果开发者使用 RS624X 系列芯片，不能使用硬件角度校准。

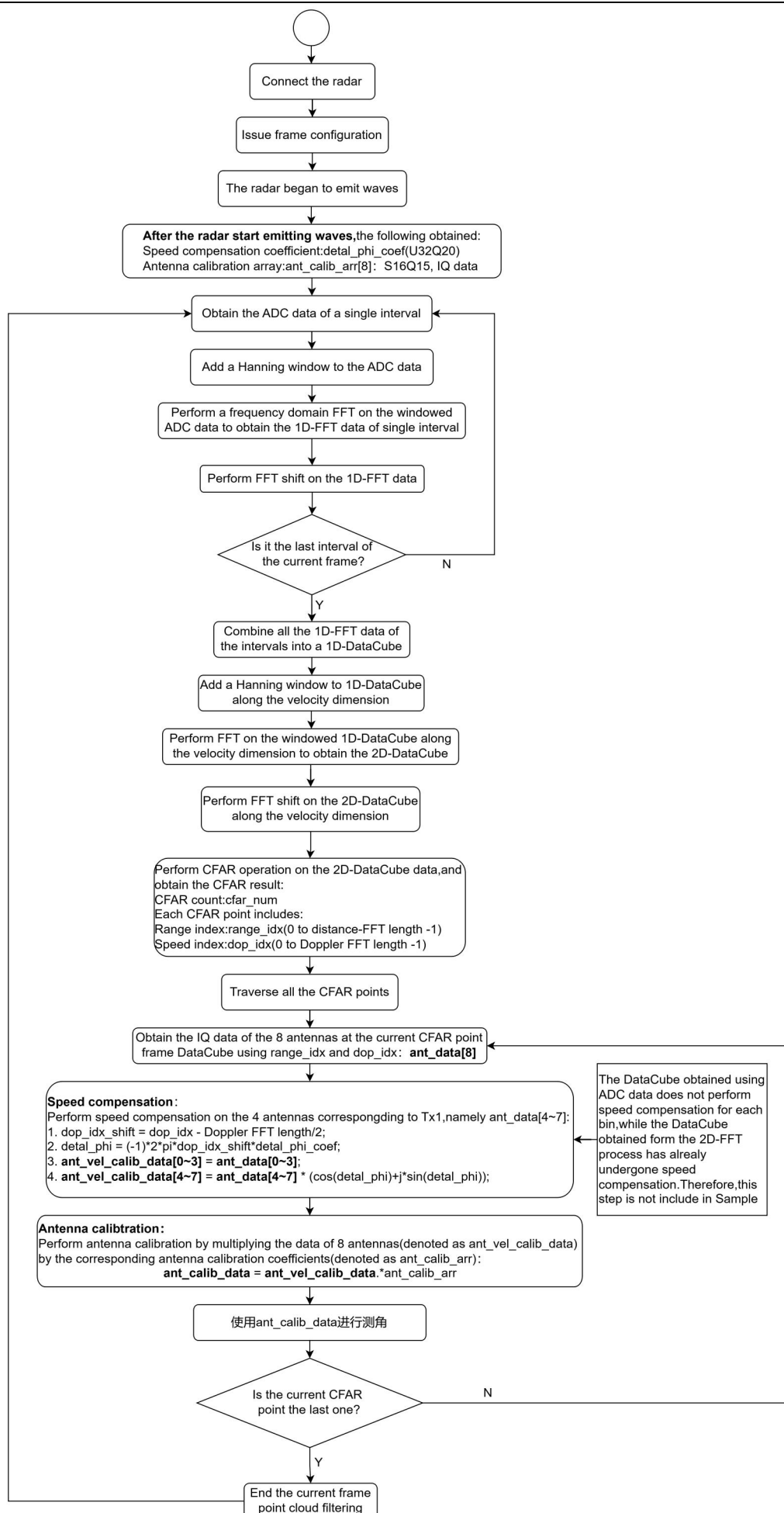


图 6-1 ADC 数据点云计算流程图

免责声明和著作权声明

本司在本文档中提供的信息力求准确，但并不保证内容完全无误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或不遵守本文档使用说明而进行的任何不当使用行为（包括但不限于如超压，超频，超温使用）所造成的损失，本司概不负责。

本文中的内容，包括但不限于供参考的 URL 地址，由于产品版本升级或其他原因，与实际内容存在差异，如有变更，恕不另行通知。文档“按现状”提供，不负任何担保责任，包括对适销性、适用于特定用途或非侵权性的任何担保，和任何提案、规格或样品在他处提到的任何担保。

本文档不负任何责任，包括使用本文档内信息产生的侵犯任何专利权行为的责任。本文档在此未以禁止反言或其他方式授予任何知识产权使用许可，不管是明示许可还是暗示许可。

版权所有©2026 珠海正和微芯科技有限公司。保留所有权利。